



Practical Course - Operating Systems - seL4 & TRENTO

W5 – TRENTO

Agenda

- Learning Goals
- TRENTOS SDK Structure
 - Content
 - Directory Structure
 - Documentation
- Getting Started
- Overview of existing TRENTOS SDK
 - Build System
 - API (Libraries)
 - Components
 - Tools
- How to create a TRENTOS application

Learning Goals

- The overall idea of the TRENTOS SDK
 - seL4 as the kernel
 - CAMkES as framework abstracting from the kernel
 - the TRENTOS SDK as the OS environment building upon this foundation
- The structure of the TRENTOS SDK (e.g. directories, files)
- How to get started with the TRENTOS SDK
 - install development & test environment (e.g. docker)
 - install the containers (build & test)
 - use the containers (e.g. for building and testing the demos)

Learning Goals ctd.

- The TRENTOS SDK internals
 - Build system (e.g. `build.sh`) & runtime environment (QEMU & proxy vs. RPi 3 B+)
 - Libraries (everything included in `sdk/libs`)
 - Demos (everything included in `sdk/demos`)
- the interaction between seL4/CAMkES and TRENTOS
- the creation of an own TRENTOS application
- the potential next steps (e.g. modifying the IoT demo)

TRENTOS SDK Structure - Content

From a high-level point of view, the SDK provides all the tools required to build and run a custom TRENTOS based system:

- a suitable Docker-based working environment containing a Linux system that provides all tools required for
 - development (e.g. compiler toolchain for ARM) and
 - testing (e.g. QEMU)
- Different runtime environments based on:
 - Emulated hardware
 - QEMU (providing the required hardware platform) and
 - the proxy application (providing required network and storage backend)
 - Physical hardware
 - RPi 3 B+ with SPI-based NOR flash memory as a storage backend
 - Nitrogen6_SoloX with two network ports
 - BD-SL-i.MX6

TRENTOS SDK Structure - Content ctd.

- the seL4 & CAMkES source code (as TRENTOS foundation)
- the source code for all TRENTOS core system libraries and components
 - exemplary driver setup suitable for the Xilinx Zynq-7000 platform (QEMU)
 - network driver
 - storage driver
 - different drivers for RPi3B+ and NXP i.MX6 platforms (e.g. network, storage)
 - network stack (PicoTCP based)
 - file system facilities (FAT, SPIFFS and LittleFS)
 - logging
 - configuration
 - crypto (cryptoserver, keystore, TLS, certificate parser)
 - various helper libraries

TRENTOS SDK Structure - Content ctd.

- the necessary firmware and bootloader files for the RPi 3 B+ and the i.MX6 SBCs
- scripts for building and running (foundation for custom projects)
- documentation in the form of
 - the TRENTOS Handbook
 - the TRENTOS API documentation (Doxygen based)
- Examples
 - Demo Hello World (simple system that prints the "hello world!" message)
 - Demo IoT App (IoT use case that uses most of the components and APIs, either on QEMU, the RPi 3 B+ or the Nitrogen6_SoloX and BD-SL-i.MX6 platforms)
 - Demo Network Filter (simple application-layer network filter utilizing two Ethernet ports, either on QEMU or the Nitrogen6_SoloX)
 - Demo TLS API (showcasing the TRENTOS TLS API by retrieving a HTTPS web page)

TRENTOS SDK Structure - Directory Structure

- **GettingStarted.pdf** → Link to `sdk/doc/pdf/TRENTOS_GettingStarted_SDK_V1.3.pdf`
- **docker** → Containers with toolchain to build and test TRENTOS systems
 - `trentos_1.3.sha256sums`
 - `trentos_build_20210503.bz2`
 - `trentos_test_20211025.bz2`
- **sdk**
 - `build-system.sh` → Master build script for building TRENTOS systems
 - `CMakeLists.txt` → Master CMake script for building TRENTOS systems
 - `README.License` → License information
 - **bin** → TRENTOS tools binaries for Linux
 - `cpt` → Configuration Provisioning Tool
 - `proxy_app` → Proxy Application
 - `rdgen` → RamDisk Generator
 - **components** → TRENTOS components sources
 - `CertServer` → Component to offer "read access" to the set of trusted certificates (the "trusted chain")
 - `ChanMux` → Channel Multiplexer component for communication with the Proxy Application
 - `CryptoServer` → CryptoServer+Keystore component
 - `EntropySource` → Dummy TRNG driver
 - `NetworkStack_PicoTcp` → Network stack component based on picoTCP
 - `NIC_ChanMux` → Network driver for a ChanMux NIC channel
 - `NIC_Dummy` → Dummy network driver
 - `NIC_iMX6` → Network driver for the iMX6 platform
 - `NIC_RPi` → Network driver for RPi3B+
 - `RamDisk` → RAM-based storage driver
 - `RPi_SPI_Flash` → Storage driver for SPI-based NOR flash memory for RPi3B+
 - `SdHostController` → Driver that implements the storage interface and allows accessing the SD card peripheral
 - `Storage_ChanMux` → Storage driver for a ChanMux storage channel
 - `StorageServer` → Providing access to storage for various clients
 - `TimeServer` → Providing access to timers for various clients
 - `TlsServer` → TLS client + networking component
 - `UART` → UART driver
 - **demo**
 - `demo_hello_world` → Hello World demo
 - `demo_iot_app` → IoT Demo (QEMU)
 - `demo_iot_app_imx6` → IoT Demo (Nitrogen6_SoloX, BD-SL-i.MX6)
 - `demo_iot_app_rpi3` → IoT Demo (RPi3B+)
 - `demo_network_filter` → Network Filter Demo (QEMU, Nitrogen6_SoloX)
 - `demo_tls_api` → Demo retrieving an HTTPS web page
- **doc**
 - `html` → TRENTOS API Doxygen documentation
 - `pdf` → TRENTOS documentation
 - `TRENTOS_GettingStarted_SDK_V1.3.pdf`
 - `TRENTOS_Handbook_SDK_V1.3.pdf`
 - `TRENTOS_ReleaseNotes_SDK_V1.3.pdf`
 - `TRENTOS_MigrationNotes_SDK_V1.2_to_V1.3.pdf`
 - `3rd_party` → Supplemental documents
- **libs** → TRENTOS libraries and core system sources
 - `3rdParty` → 3rd party libraries used by TRENTOS libraries internally
 - `chanmux` → ChanMux
 - `chanmux_nic_driver` → Network driver based on ChanMux
 - `lib_compiler` → Compiler abstraction utilities
 - `lib_debug` → Debugging facility macros and functions
 - `lib_host` → Host side utilities
 - `lib_io` → I/O operations
 - `lib_logs` → Logging utilities
 - `lib_macros` → Commonly used macros
 - `lib_mem` → Memory management utilities
 - `lib_osal` → Operating System abstraction library
 - `lib_server` → Server components utilities
 - `lib_utils` → Common useful utilities
 - `os_cert` → Certification parser library for x509 certificates
 - `os_configuration` → Configuration server library
 - `os_crypto` → Crypto library
 - `os_filesystem` → File system library (FAT, SPIFFS, LittleFS)
 - `os_keystore` → Keystore library (file based, RAM based)
 - `os_logger` → Log server core library
 - `os_socket` → Socket library
 - `os_tls` → TLS client library
- **os_core_api** → TRENTOS APIs for all subsystems
- **resources**
 - `nitrogen6sx_sd_card` → SD card files with Nitrogen6_SoloX bootloader
 - `openocd_cfgs` → Configuration files for debugging with OpenOCD
 - `rpi3_sd_card` → SD card files with RPi3B+ bootloader
 - `sabre_sd_card` → SD card files with BD-SL-i.MX6 bootloader
- **scripts** → Utility scripts
- **sdk-seL4-canKes** → seL4 and CamKes
- **tools** → TRENTOS tools source code
 - `cpt` → Configuration Provisioning Tool
 - `proxy` → Proxy Application
 - `rdgen` → RamDisk Generator
 - `rpi3_flasher` → TRENTOS helper system for flashing an image on the RPi3B+ SPI-based NOR flash memory

TRENTOS SDK Structure - Documentation

The TRENTOS SDK basically contains four types of documentation:

- The TRENTOS Getting Started Document
- The TRENTOS Handbook in PDF form
- The TRENTOS API documentation (Doxygen based)
- 3rd-party background information about the seL4/CAmkES ecosystem (scientific papers from UNSW/Data61)

TRENTOS SDK V1.3

The TRENTOS Handbook

TRENTOS SDK Structure - Getting Started

- Docker based development & testing

- Build container (e.g. compiler, ...)

```
# execute a specific script in a new trentos_build container
sdk/scripts/open_trentos_build_env.sh <path_to_build_script>
```

- Test container (e.g. QEMU, ...)

```
# execute a specific script in a new trentos_test container
sdk/scripts/open_trentos_test_env.sh <path_to_test_script>
```

- Docker will bind the current working folder to a volume mounted under */host* in the *trentos_build* container, execute the build and then remove the container again. Then the script is executed in the container and the container is removed once the script finishes.

- Setup

- Project directory
- Build platform
- Output directory
- Build type

```
sdk/scripts/open_trentos_build_env.sh \
  sdk/build-system.sh \
  sdk/demos/demo_hello_world \
  zynq7000 \
  build-zynq7000-Debug-demo_hello_world \
  -DCMAKE_BUILD_TYPE=Debug
```

TRENTOS SDK Structure - Getting Started ctd.

- Similar to the build process, the *trentos_test* container will
 - bind the current working folder to a volume mounted under /host in the container
 - execute the run script and then remove the container again
- Additionally: the *trentos_test* docker container automatically sets up the network interface and configures access permission

```
# change to the build output folder
cd build-zynq7000-Debug-demo_hello_world
```

```
# execute the Hello World demo in the trentos_test container
../sdk/scripts/open_trentos_test_env.sh ./simulate
```

```
Booting all finished, dropped to user space
main@main.c:2125 Starting CapDL Loader...
main@main.c:2127 CapDL Loader done, suspending...
hello world!
```

TRENTOS SDK Structure - Running on the RPi 3 B+

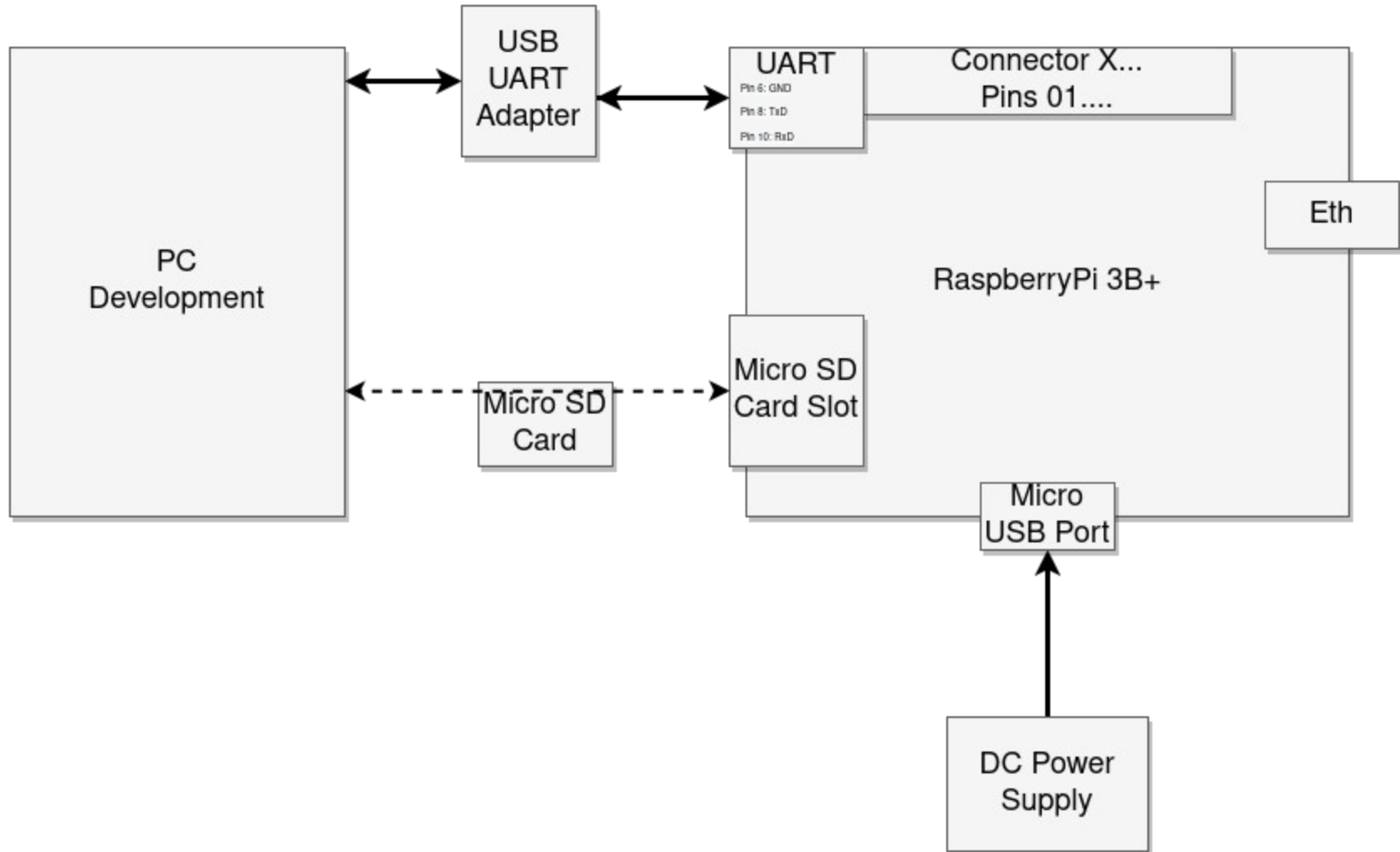
- TRENTOS on the RPi 3 B+
 - build for the RPi 3 B+ hardware platform
 - prepare the SD card with necessary files
 - copy the RPi 3 B+ firmware blobs (*bootcode.bin*, ...)
 - copy the TRENTOS system image (*os_image.elf*)
 - Connect the PC and the RPi 3 B+ via USB-to-UART adapter and start serial communication (e.g. picocom)
 - Terminal output shows errors or success

```
U-Boot 2020.07-rc3-00002-g4b6f96a0a6 (May 26 2020 - 18:38:13 +0200)
```

```
...
```

```
Bootstrapping kernel  
Booting all finished, dropped to user space  
main@main.c:2125 Starting CapDL Loader...  
main@main.c:2127 CapDL Loader done, suspending...  
hello world!
```

TRENTOS SDK Structure - Running on the RPi 3 B+



TRENTOS SDK Buildsystem

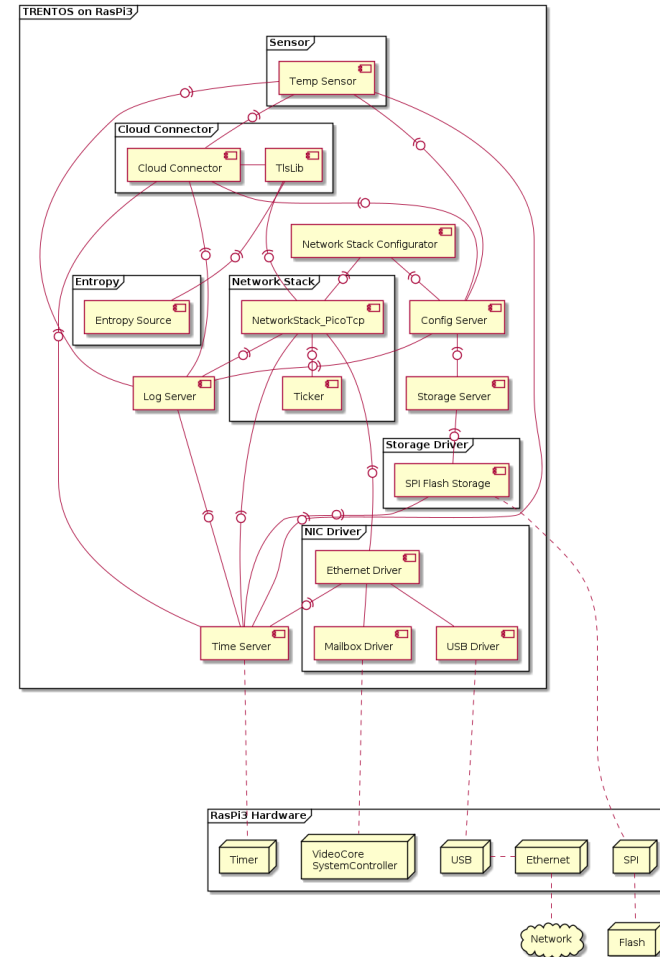
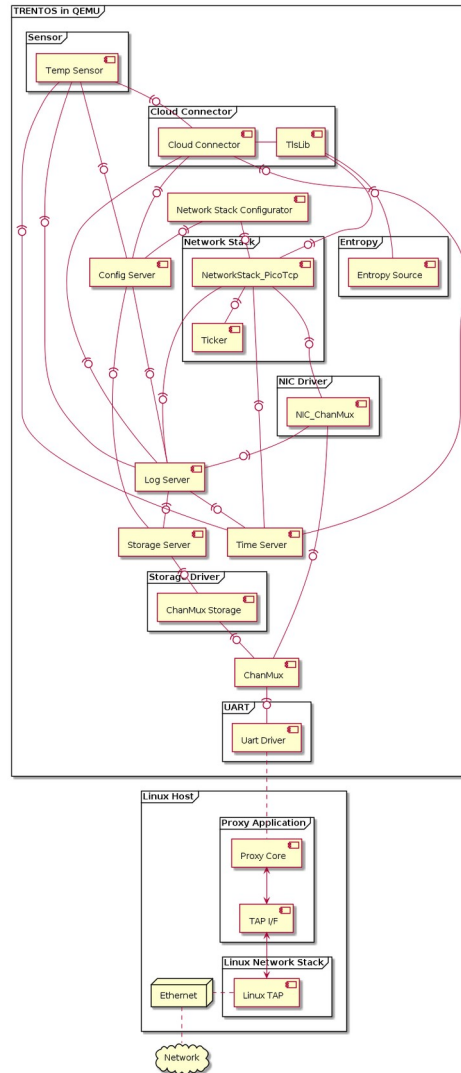
- The TRENTOS build system uses CMake/ninja and is based on the seL4/CAmkES build system
- The master build script is *build-system.sh*, which supports an out-of-source build for arbitrary OS projects
 - <SDK> is the path to the SDK location of the script
 - <OS_PROJECT_DIR> is the path to the OS project to build
 - <BUILD_PLATFORM> is the target platform
 - <BUILD_DIR> is the folder where the build output will be created in
 - -D CMAKE_BUILD_TYPE=<Debug|Release|...> is a CMake parameter specifying the type of the build
 - any additional parameters will be passed to CMake

```
<SDK>/build-system.sh
    <OS_PROJECT_DIR>
    <BUILD_PLATFORM>
    <BUILD_DIR>
    -DCMAKE_BUILD_TYPE=<Debug|Release|RelWithDebInfo|MinSizeRel>
    ...
```


TRENTOS SDK Buildsystem

- The script *build-system.sh* internally invokes CMake with the file *<SDK>/CMakeLists.txt*
- The CMake build performs the following steps:
 - set up the build environment for seL4/CAMkES using *<SDK>/sdk-sel4-camkes/helper.cmake*
 - build the seL4 microkernel
 - build the CAMkES tools
 - includes a custom file *<OS_PROJECT_DIR>/CMakeLists.txt* via the CMake function *add_subdirectory()* that defines the custom OS system to build
 - The system's build output will then be in the folder *<build-directory>/os_system*
- demo systems shipped with the SDK provide examples of content of *<OS_PROJECT_DIR>/CMakeLists.txt*

TRENTOS - Overview



TRENTOS Components - Overview

- TRENTOS standard components already implement logic to be re-used on several occasions
- Components
 - (partially) implement the TRENTOS API
 - provide easier access to (specific) TRENTOS API
 - provide a reference implementation for several TRENTOS API use cases

TRENTOS Components - Overview ctd.

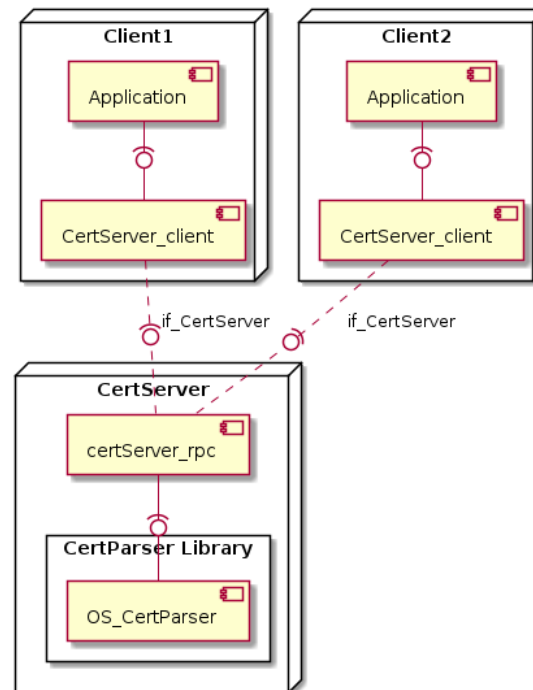
- Crypto
 - EntropySource
 - CertServer
 - CryptoServer
 - TlsServer
- ChanMux (+ UART)
- Network Stack
- Driver
 - Network (Dummy, ChanMux, RPi 3 B+, i.MX6)
 - Storage (RamDisk, ChanMux, RPi SPI Flash, SdHostController)
- Misc
 - StorageServer
 - TimeServer

TRENTOS Components - EntropySource

- Implements a platform-specific driver for providing entropy from a hardware-based true random number generator (TRNG) and offers access to this hardware via a defined interface
- A good source of entropy is critical for any application that wants to use cryptography (e.g. to generate keys)
- Offers the *if_OS_Entropy* interface
- Other components can use this interface to request raw entropy
- Info: Currently, TRENTOS does not provide a platform-specific driver
 - contains a simple dummy implementation
 - the real implementation of this component is specific to each target platform, depending on the availability of actual hardware modules which can act as a source of entropy

TRENTOS Components - CertServer

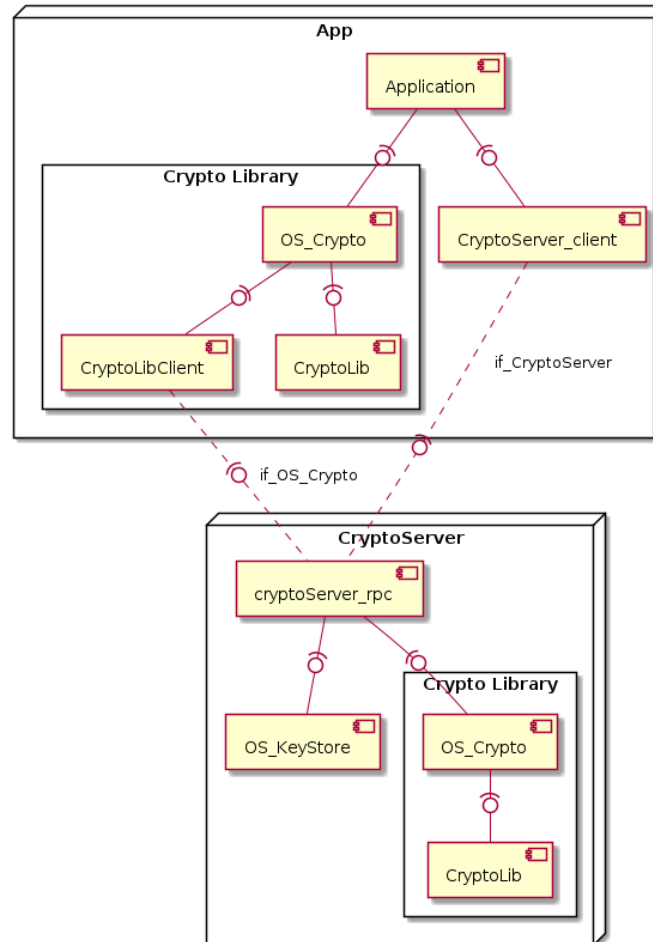
- configured with a set of trusted certificates (the "trusted chain")
- offers "read access" to these certs to its clients via a simple interface
- Idea: isolation of trusted certificates in small component (prevent from change)
- Utilizes TRENTOS CertParser API
- Interfaces
 - Provides *if_CertServer*
 - Uses *if_OS_Entropy*



TRENTOS Components - CryptoServer

- The CryptoServer encapsulates key handling (e.g. storing, loading) and crypto operations (e.g. generation of signatures, etc.) in an isolated component
- The CryptoServer component itself is trusted
 - supposed to be a central element of the TRENTOS security architecture, usable by other CAmkES components
 - used to isolate sensitive data (e.g. keys) in its own component from the rest of the system, which is potentially not trusted
- Beware: the CryptoServer uses the KeyStore for the persisting of key material
 - KeyStore currently does not support encryption or integrity protection
 - needs to be ensured that keys cannot be extracted easily (e.g. by using only internal/sealed memories, etc.)
- Utilizes TRENTOS Keystore API and TRENTOS Crypto API
 - CryptoServer uses both internally and extends them with the ownership concept for keys
 - Every client of the CryptoServer has its own instance of the Crypto API and KeyStore (no client can interfere with another client's keys or cryptographic operations)

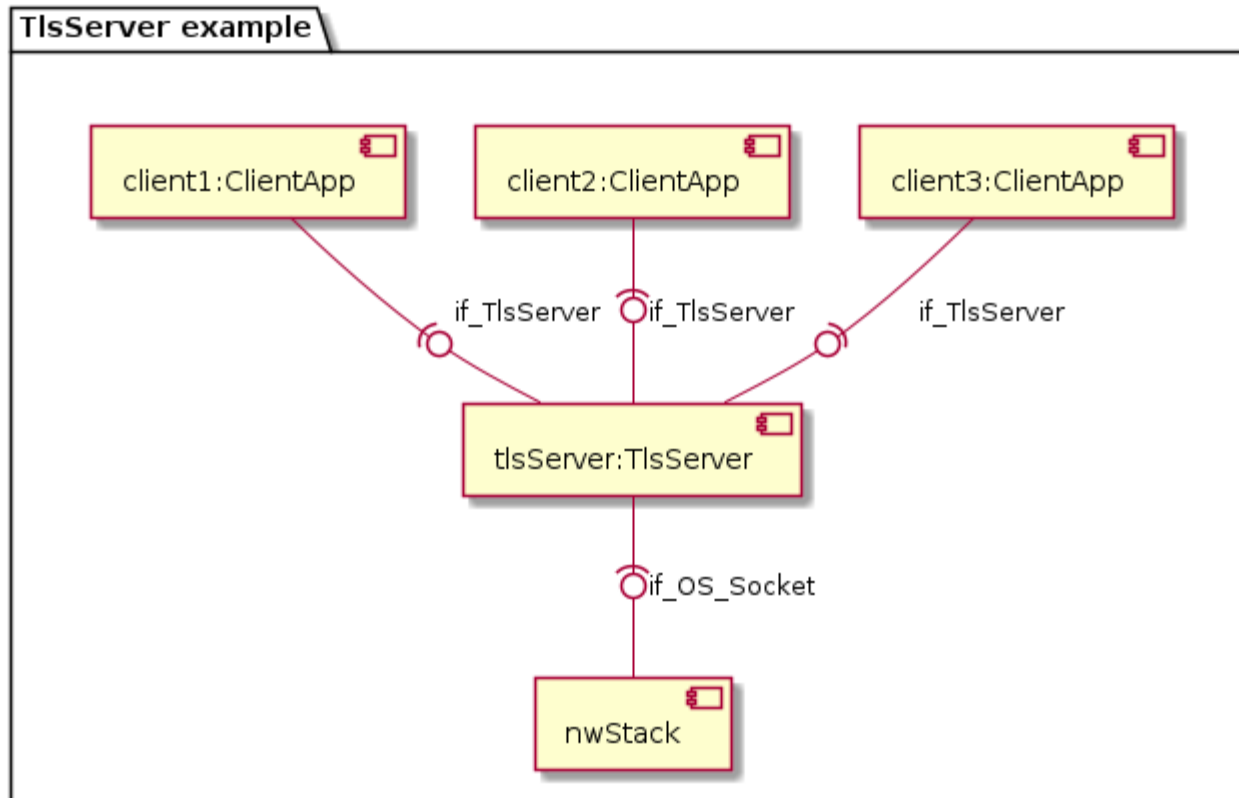
TRENTOS Components - CryptoServer ctd.



TRENTOS Components - TlsServer

- encapsulates socket connection handling (e.g. connect, disconnect) and TLS protocol operations (e.g. handshake, etc.) in an isolated component
- Info: This component does not implement a "TLS server" in the sense of being a TLS protocol endpoint with a server socket
 - "Server" is to be understood in terms of RPC communication
 - supports up to 8 clients and uses a pre-configured certificate to validate TLS connections
- In contrast to the CryptoServer, the trust relationship is inverted
 - all the complexity of the TLS protocol is put into a separate component
 - if this component is compromised, it cannot easily affect the rest of the system
- Utilizes TRENTOS Socket API and TRENTOS TLS API

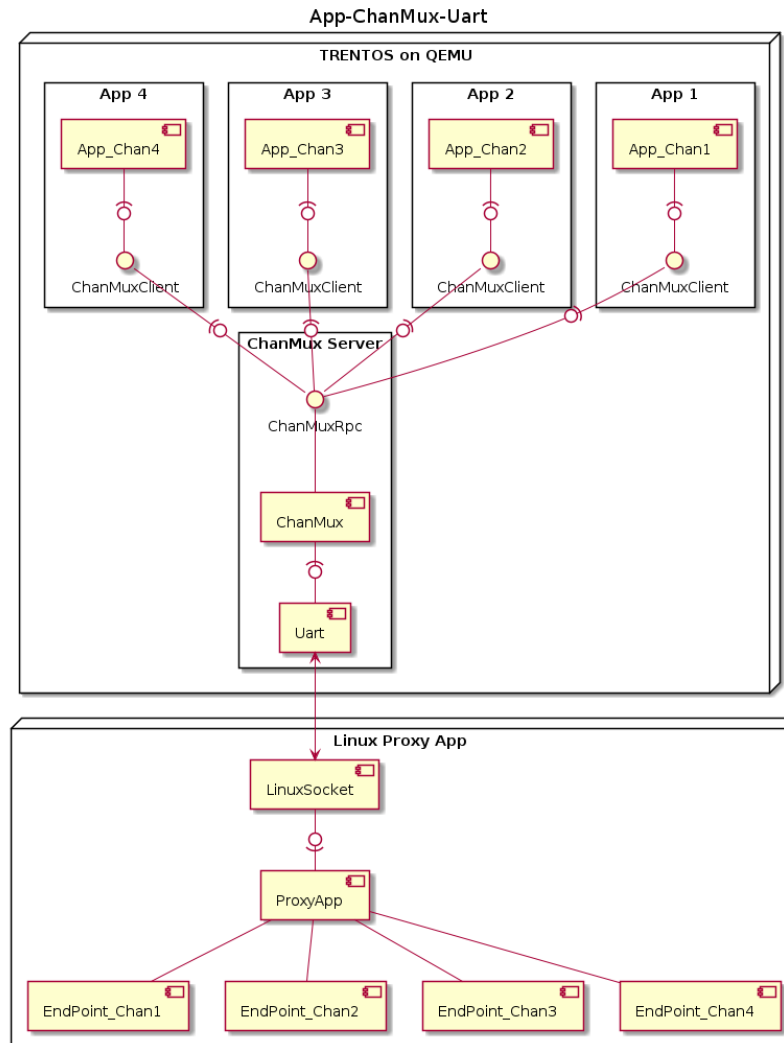
TRENTOS Components - TlsServer ctd.



TRENTOS Components - ChanMux

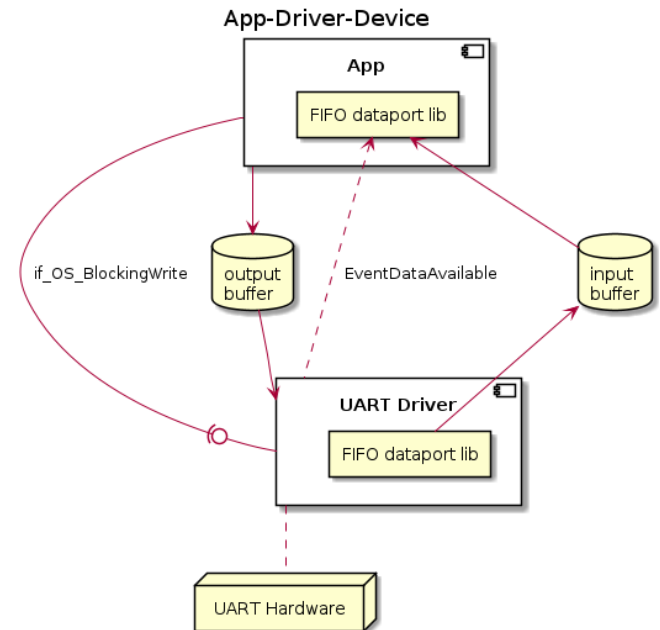
- multiplexes an arbitrary number of bidirectional full-duplex communication channels from clients over a single channel
- agnostic of the upper and lower channel details
 - lower physical channel: e.g. UART
 - lower layer protocol implements a packet-oriented data link protocol (HDLC) with a maximum packet size (MTU)
- Idea: cope with limited driver support (on new platforms)
- Within TRENTOS SDK Docker containers
 - virtualized UART (Host Bridge) via QEMU
 - TRENTOS SDK includes "Proxy" application, which is running on the host and can connect to QEMU via PTY or TCP socket
 - proxy provides a set of dedicated channels, emulating various hardware devices like persistent storage or network
 - ChanMux muxes these channels into the UART and the proxy application demuxes the channels and implements the device emulation

TRENTOS Components - ChanMux ctd.



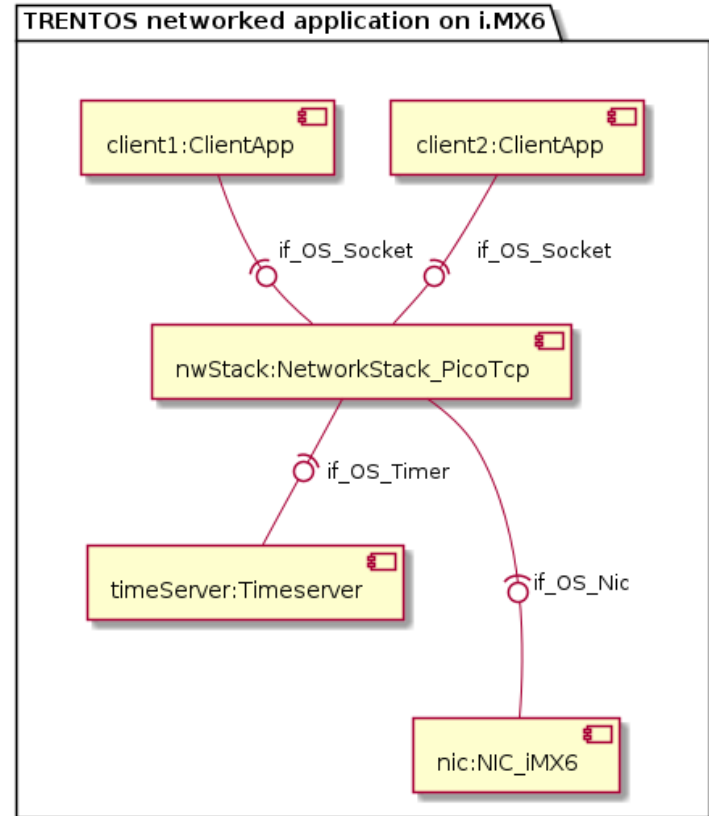
TRENTOS Components - UART

- The UART component implements a generic UART driver that can be used on various platforms
 - The component wraps the CAMkES details for creating a UART driver and connecting it to the hardware
 - The component uses the seL4 platform library code for the platform-specific low-level hardware access
- Beware: UART is used in raw mode, no additional characters are added or removed from the data stream (especially, no CR/LF line break conversion)
 - Purpose: binary data transfer, currently not intended for console usage
 - Hint: for log output, the LogServer component should be used



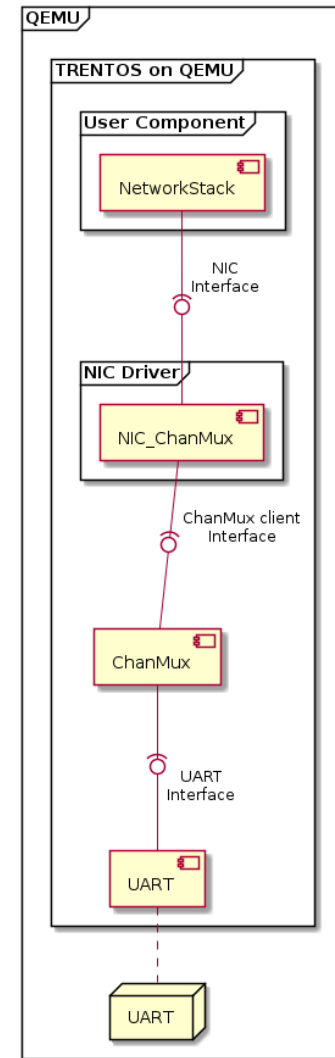
TRENTOS Components - NetworkStack_PicoTcp

- Network access is realized via sockets
- Implements the IP stack functionality and offers TCP and UDP network connectivity to any user application connected to it
 - implements the TRENTOS Socket API
 - provides the *if_OS_Socket* interface
 - uses the *if_OS_Nic* interface (towards a NIC driver)
 - offers a wrapper around the picoTCP stack
- Either one network stack instance can be used by multiple applications or each application can use a dedicated network stack instance



TRENTOS Components - Network Driver

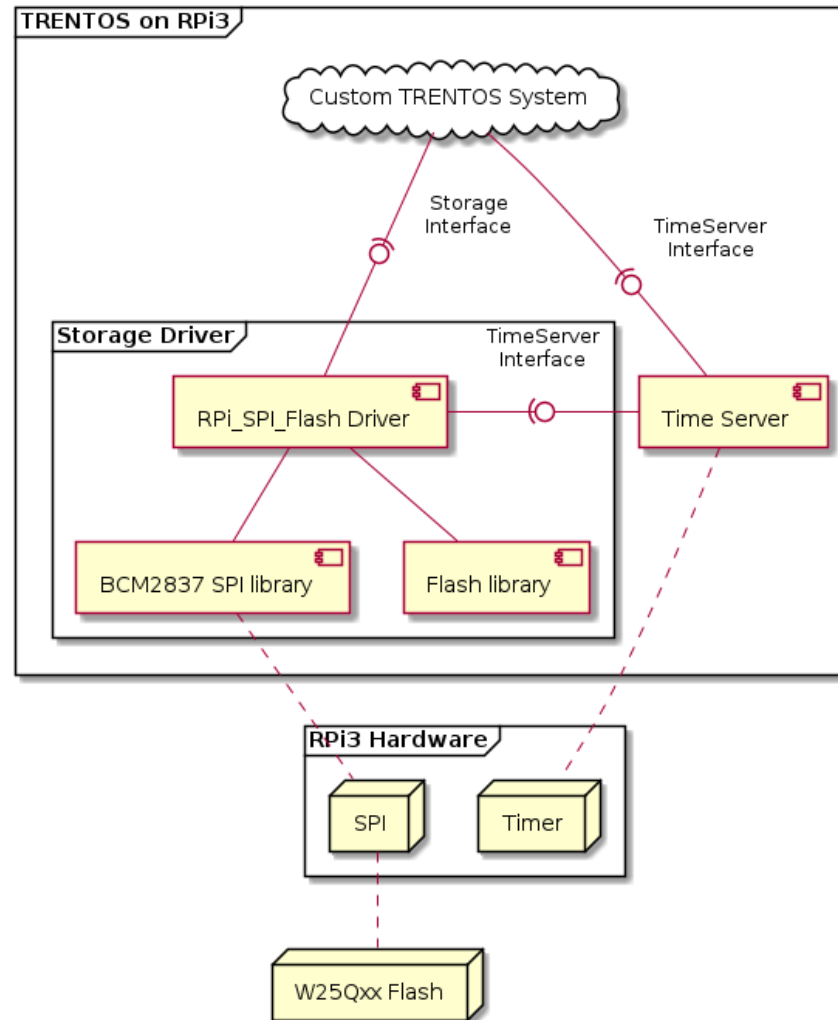
- Driver types
 - NIC_Dummy: does not implement any functionality, just acts as a dummy network interface for development purposes (e.g. prototyping)
 - NIC_ChanMux: wraps a ChanMux NIC Channel (which actually is a pair of a "ctrl" and a "data" channel)
 - NIC_iMX6: provides Ethernet functionality for NXP's i.MX6 family of SOCs (currently restricted to i.MX6Quad and i.MX6SoloX)
 - NIC_RPi: allows using the network interface on the Raspberry Pi 3 B+
- Expose the TRENTOS *if_OS_Nic* interface



TRENTOS Components - Storage Driver

- Driver types
 - RamDisk: maps all I/O functionality to an internal buffer in RAM
 - Storage_ChanMux: wraps a ChanMux Storage Channel (e.g. storing data on the QEMU host)
 - RPi_SPI_Flash: a driver for a NOR flash device (e.g. W25Q64) connected to the Raspberry Pi's SPI interface
 - SdHostController: allows accessing the SD card peripheral on i.MX6 based SBCs and the RPi 3 B+
- Expose the TRENTOS *if_OS_Storage* interface

TRENTOS Components - Storage Driver ctd.



TRENTOS Components - Misc

- StorageServer
 - As seen before: storage can be generic (e.g. a RamDisk) or platform-specific (e.g. a SPI flash driver)
 - enables multiple client components to access a storage device
 - Storage is divided into different segments (kind of „partitions“) which can then be accessed by the clients of the StorageServer
 - Exposes the TRENTOS *if_OS_Storage* interface (both to the upper and lower layer)
- TimeServer
 - uses a hardware timer
 - allows multiple clients to share a hardware timer for common functions such as sleep, periodic notifications or getting a timestamp
 - uses the platform-specific timer modules from the seL4 platform support library
 - Exposes the TRENTOS *if_OS_Timer* interface

TRENTOS Components - How to use

- Declaration of the component in CMake (within a project's *CMakeLists.txt* file)
 - Some component's require specific parameters added to LIBS (e.g. working as client or server)
- Instantiation and Configuration in CAmkES (within a project's main .camkes file)
 - Declaration of the component
 - Instantiation of the component
 - Connection of the component (e.g. to depending services, to clients)
 - Optional: Configuration of the component instance (+ badge assignment)

TRENTOS Components - How to use (example)

```
StorageServer_DeclareCamkesComponent(
    StorageServer
)

DeclareCamkesComponent(
    TestApp
    SOURCES
        components/TestApp/src/TestApp.c
    C_FLAGS
        -Wall
        -Werror
    LIBS
        system_config
        lib_debug
        os_core_api
        os_filesystem
)
```

```
import <std_connector.camkes>;

#include "system_config.h"

//-----
// Storage
//-----

#include "RamDisk/RamDisk.camkes"
RamDisk_COMPONENT_DEFINE(RamDisk)

#include "StorageServer/camkes/StorageServer.camkes"
StorageServer_COMPONENT_DEFINE(StorageServer)

import "components/TestApp/TestApp.camkes";

//-----
// System
//-----

assembly {
    composition {
        //-----
        // Storage
        //-----
        component RamDisk ramDisk;

        //-----
        // StorageServer
        //-----
        component StorageServer storageServer;

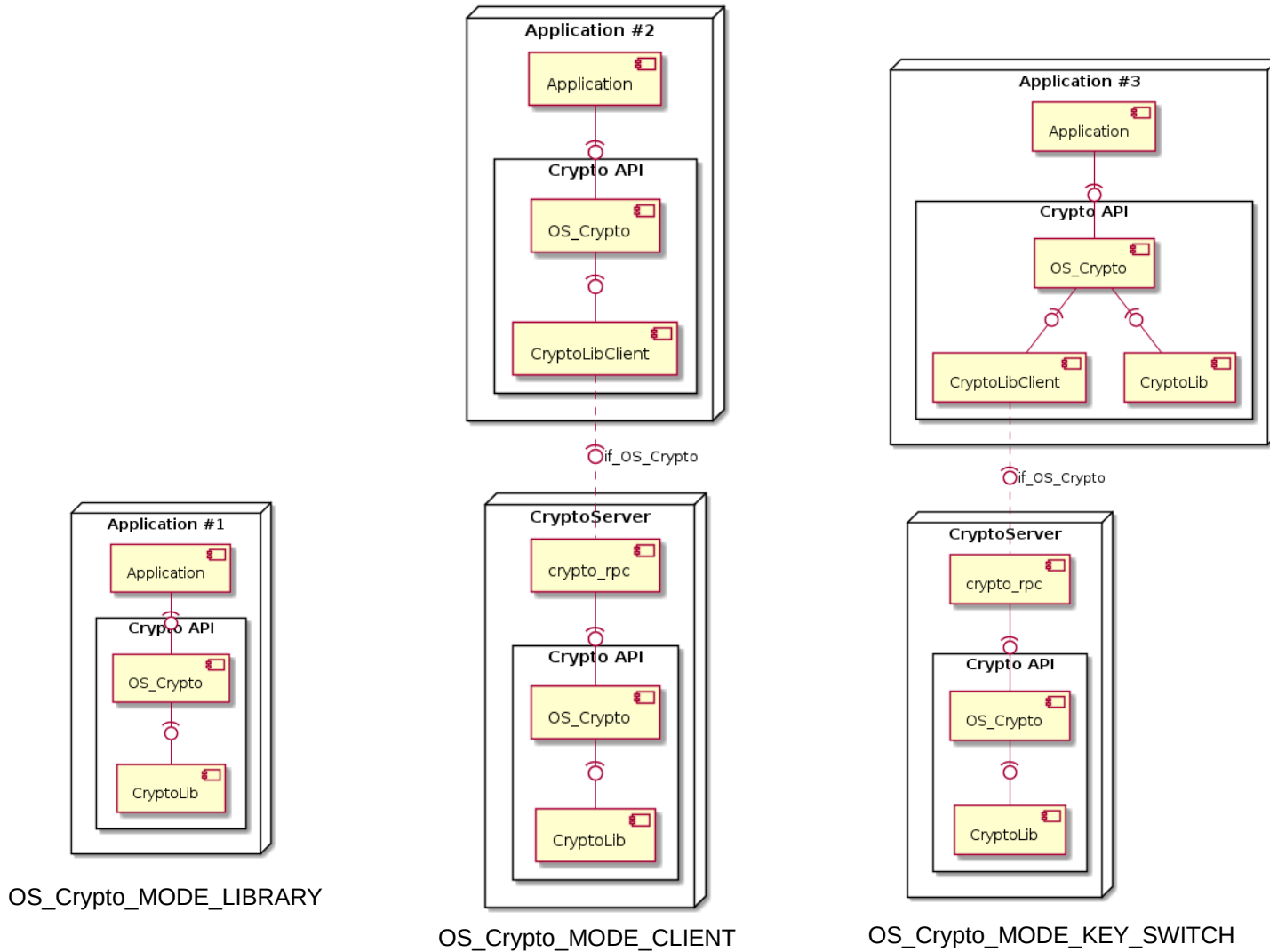
        StorageServer_INSTANCE_CONNECT(
            storageServer,
            ramDisk.storage_rpc, ramDisk.storage_port
        )

        StorageServer_INSTANCE_CONNECT_CLIENTS(
            storageServer,
            testApp.storage_rpc_1, testApp.storage_dp_1,
            testApp.storage_rpc_2, testApp.storage_dp_2
        )
    }
}
```

TRENTOS API - Crypto

- TRENTOS Crypto API (supports a set of cryptographic operations via a generic interface)
 - can generate keys, random numbers, ...
 - requires access to a good source of entropy
- TRENTOS TLS API (simple creation and usage of TLS client connections according to the TLS 1.2 standard)
- TRENTOS CertParser API (functionality for parsing and validating x509 certificates)
 - uses the TRENTOS Crypto API to perform signature verification and hash calculation
 - uses the x509 parser code provided by mbedTLS
- TRENTOS Keystore API (allows to persist and load keys used by the TRENTOS Crypto API via an instance of the TRENTOS FileSystem API)
 - allows to store and load keys identified by memorable names
 - currently two different Keystore implementations available (KeystoreFile using a file based storage backend and KeystoreRamFV, which is formally verified and stores the keys in RAM)

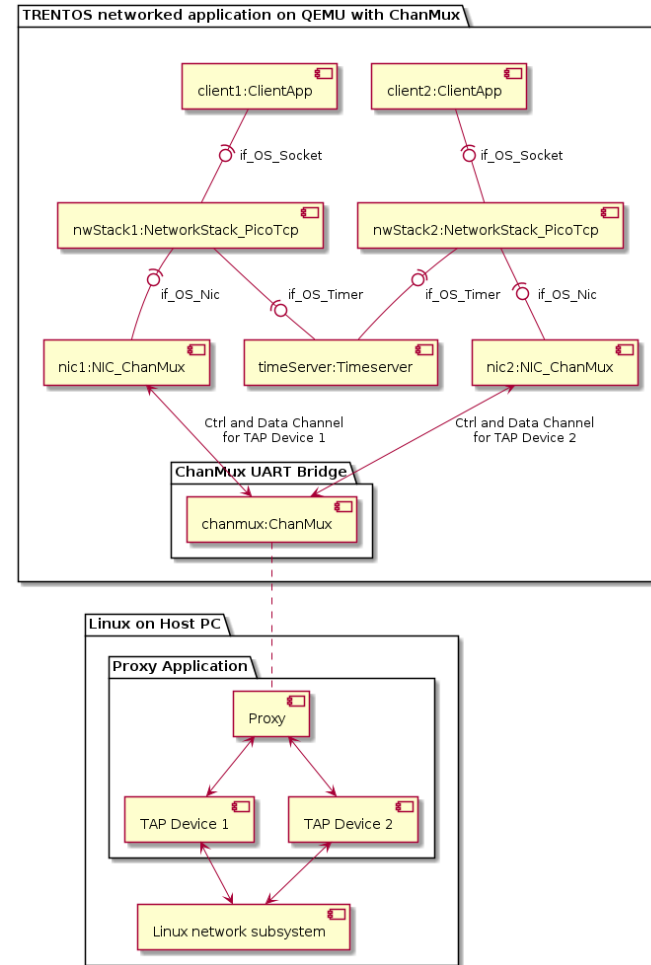
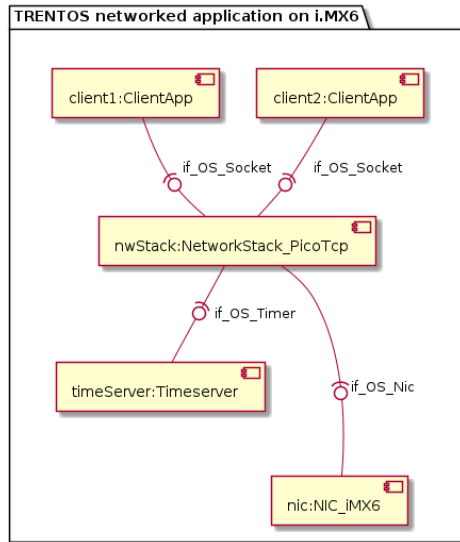
TRENTOS API - Crypto ctd.



TRENTOS API - Socket & Storage

- **TRENTOS FileSystem API** (supports the creation of file systems of various types and typical file operations)
 - provides access to typical file system operations based on an abstract concept of "storage"
 - any component that provides the *if_OS_Storage* interface can be used through the FileSystem API
 - file system library is effectively a wrapper around third-party file system implementations (currently FAT, SPIFFS and LittleFS)
- **TRENTOS Socket API** (provides functions to establish and use TCP and UDP communication channels)
 - includes client-side function implementations to communicate with a network stack component via the *if_OS_Socket* interface
 - implemented in a non-blocking way
 - currently supports IPv4 connections only

TRENTOS API - Socket & Storage ctd.



TRENTOS API - Misc

- TRENTOS ConfigService API
 - a centralized configuration database
 - can store the settings for any application or system component that opts to use this service
 - offers a standardized way of storing and preparing all the relevant settings in one place
- TRENTOS Logger API
 - an abstraction of the logging functionality
 - allows setting up a logger server to which clients can send logging entries

TRENTOS API - How to use

- Select API-specific instantiation mode, e.g. for Crypto API
 - `OS_Crypto_MODE_LIBRARY`
 - `OS_Crypto_MODE_CLIENT`
 - `OS_Crypto_MODE_KEY_SWITCH`
- Instantiation of API in CMake
 - link target with the appropriate build target (e.g. *os_crypto*)
 - done within LIBS part of `DeclareCAmkESComponent` in project's *CMakeLists.txt* file
- Using the API in C
 - import respective header file (e.g. *OS_Crypto.h*)
 - provide respective configuration (e.g. using Crypto API in local mode as library via specifier `OS_Crypto_MODE_LIBRARY`)
 - use appropriate API functions

TRENTOS API - How to use (example)

```
// For Crypto API
#import "OS_Crypto.h"

// For RPCs and dataports
#include <camkes.h>

// Configuration of Crypto API in "LIBRARY" mode; assign entropy source
// as defined in CAMKES..
static OS_Crypto_Config_t cryptoCfg = {
    .mode      = OS_Crypto_MODE_LIBRARY,
    .entropy   = IF_OS_ENTROPY_ASSIGN(
        entropy_rpc,
        entropy_port)
};

int run()
{
    // Definition of a key-generation spec for a 128-bit AES key
    static OS_CryptoKey_Spec_t aes128Spec = {
        .type = OS_CryptoKey_SPECTYPE_BITS,
        .key = {
            .type      = OS_CryptoKey_TYPE_AES,
            .params.bits = 128
        }
    };
    // Declare handles for API and objects
    OS_Crypto_Handle_t hCrypto;
    OS_CryptoKey_Handle_t hKey;
    OS_CryptoCipher_Handle_t hCipher;
    // Declare inputs and outputs for crypto operation
    uint8_t pt[16] = "0123456789abcdef";
    uint8_t ct[16];
    size_t ptLen = 16, ctLen = 16;

    // Init Crypto API instance in ROUTE mode
    OS_Crypto_init(&hCrypto, &cryptoCfg);

    // Generate a new AES key based on the spec provided above
    OS_CryptoKey_generate(&hKey, hCrypto, &aes128Spec);
}
```

TRENTOS Tools

- TRENTOS Configuration Provisioning Tool
 - creates provisioned binary files that contain configuration data that can be managed by the ConfigService library
 - configuration data is first parsed from a XML file, interpreted and serving connected clients with their requested configuration parameters
- TRENTOS RamDisk Generator Tool (used to convert a file system image produced by the Configuration Provisioning Tool into a file that can be linked into the RamDisk to pre-provision it)
- TRENTOS Flasher Tool for RPi 3 B+ (used to dump a previously created RamDisk image from the RamDisk Generator Tool to the SPI flash of a RPi 3 B+)
- TRENTOS Proxy Application
 - provides a Linux host backend for a TRENTOS instance running within a QEMU environment
 - data exchange is done with the TRENTOS ChanMux component running via PTY or TCP endpoint

TRENTOS - Create own application

- Each application consists of
 - Build/test specific parts (*CMakeLists.txt*, *run_demo.sh*)
 - A central CAMkES application description (main *.camkes* file), which is used for declaration and instantiation of all required components and connections
 - re-using TRENTOS standard components (by utilizing specific macros)
 - creating own components (e.g. making use of the TRENTOS API)
 - A folder „components“ which contains a folder for each own component offering
 - a *.camkes* component description + optional interface definitions
 - own logic in C code + usage of TRENTOS API
 - Optional: a global configuration (*system_config.h*)
- **Note:** a detailed step-by-step guide will be offered in form of the homework assignment

Sebastian Eckl, M.Sc.
Senior Software Engineer
sebastian.eckl@hensoldt-cyber.de

HENSOLDT Cyber GmbH
Willy-Messerschmitt-Straße 3
82024 Taufkirchen
www.hensoldt-cyber.com

